

Rapport Soutenance

- Projet RUST -

Équipe:

Ayoub El-Asri - Implémentation algorithmique

Lucas Lefranc - Interface utilisateur

Maywenn Koussawo - Designer & web

Date : 19 / 05/ 2026

Table des Matieres

1 - Introduction

2 - Le rôle et la mission de chaque membre

2.1 - Lucas Lefranc

2.2 - Ayoub El-Asri

2.3 - Maywenn Koussawo

3 - Objectifs, planification des tâches et avancement du projet pour chaque membre

4 - Difficultés rencontrées et précisions

5 - Conclusion

1 - Introduction

Le projet **KOMP** s'inscrit dans le cadre du semestre 3 à l'**Epita**. Il a pour objectif de concevoir une application de traitement linguistique avancée capable de corriger, d'analyser et de comparer des textes de manière automatisée, en utilisant des techniques de **NLP (Natural Language Processing)** et d'algorithmique textuelle.

Concrètement, l'application doit être en mesure de :

- **Charger ou saisir un texte** brut au sein d'une interface dédiée ;
- **Analyser la syntaxe et l'orthographe** pour identifier les erreurs contextuelles complexes (homophones, accords du participe passé) ;
- **Proposer des corrections intelligentes** accompagnées d'explications pédagogiques pour l'utilisateur ;
- **Comparer deux versions d'un document** (module *Compare*) via un algorithme de "diffing" pour visualiser instantanément les modifications effectuées ;
- **Adapter le niveau de correction** selon le registre de langue souhaité (formel, académique ou courant) ;
- **Afficher et sauvegarder** le résultat final sous différents formats (PDF, TXT).

Ce projet mobilise plusieurs compétences clés : algorithmique avancée, programmation structurée en Langage, **ex: C ou Python**, gestion de projet en équipe et conception d'interfaces graphiques (GUI). Chaque membre du groupe est responsable d'un module spécifique (moteur de correction, algorithme de comparaison, ou interface), dont l'intégration fluide est la clé de la chaîne de traitement.

Objectifs de la soutenance finale

L'objectif de ce rapport est de documenter l'évolution de notre solution **KOMP** depuis la première soutenance. L'accent sera mis sur la **robustesse du moteur de correction** face à des textes de "Niveau 2" et "Niveau 3" (contenant des ambiguïtés sémantiques fortes ou des structures grammaticales complexes) ainsi que sur l'**intégration finale** de l'algorithme de comparaison au sein d'une interface graphique utilisateur (GUI) ergonomique et performante.

2 - Le rôle et la mission de chaque membre

L'organisation du projet **KOMP** repose sur une division modulaire des tâches, permettant à chaque membre de se concentrer sur une expertise spécifique tout en assurant l'interopérabilité des composants via des APIs internes.

2.1 Lucas LEFRANC

2.1.1 Rôle et mission

Luca est responsable du module **"Interface Utilisateur (GUI)"**. Sa mission consiste à concevoir l'environnement de travail de l'utilisateur final. Étant le point d'entrée et de sortie du logiciel, la qualité de son travail détermine l'accessibilité et l'ergonomie de l'application KOMP.

Son rôle est de transformer une logique de traitement textuelle complexe en une expérience fluide. Ses responsabilités principales se décomposent comme suit :

- **Architecture de l'Interface** : Mise en place de la fenêtre principale, des zones de saisie dynamique et des menus de configuration.
- **Gestion des Événements** : Interceptor les actions de l'utilisateur (clics, raccourcis clavier) pour déclencher les appels vers le moteur de correction ou le module de comparaison.
- **Affichage des Retours** : Assurer la visualisation claire des erreurs (soulignement, infobulles) et l'affichage côte à côte des textes comparés.

2.2 Maywenn KOUSSAWO

2.2.1 Rôle et mission

Maywenn occupe le rôle de **"Web Designer & UX Specialist"**. Sa mission est de définir l'identité visuelle de KOMP et de garantir que le parcours utilisateur est logique, esthétique et moderne.

Elle travaille en étroite collaboration avec Luca pour intégrer les maquettes graphiques dans le code de l'interface. Ses responsabilités clés incluent :

- **Charte Graphique** : Création du logo, choix de la palette de couleurs et de la typographie pour maximiser la lisibilité du texte.
- **Maquetage (Wireframing)** : Conception des prototypes d'écrans avant leur développement pour tester l'ergonomie.
- **Responsive Design** : S'assurer que les composants visuels (boutons, listes déroulantes) s'adaptent à la taille de la fenêtre sans dégrader l'expérience de correction.

2.3 Ayoub EL-ASRI

2.3.1 Rôle et mission

Ayoub est responsable du module "**Algorithmes de Comparaison & Logique (Back-end)**". Sa mission est de développer le moteur de l'application, celui qui traite les données textuelles brutes pour en extraire du sens.

Il est le garant de la performance algorithmique de KOMP. Ses responsabilités se décomposent comme suit :

- **Moteur de Diffing** : Développement de l'algorithme de comparaison permettant d'identifier les différences entre deux versions d'un texte.
 - **Traitement de la donnée** : Implémentation des structures de données optimisées pour la manipulation de chaînes de caractères volumineuses.
 - **Optimisation** : Réduction de la complexité algorithmique pour assurer un traitement instantané, même sur des documents de plusieurs pages.
-

3 - État d'Avancement Final et Réalisation Technique

3.1 Avancement Final

L'objectif majeur de cette phase était de passer d'un simple moteur de recherche textuelle à une plateforme de correction et de comparaison intégrée. Cet objectif a été rempli avec succès, validant ainsi la chaîne complète de traitement de **KOMP** :

- **Implémentation du Moteur de Correction "From Scratch"** : Contrairement à l'utilisation de bibliothèques de haut niveau, le cœur de l'analyseur a été développé en **C**. Cela inclut la gestion des dictionnaires sous forme de **Hash Tables** pour garantir un temps de recherche en $O(1)$.
 - **Validation de l'Algorithme de Comparaison (Diffing)** : L'algorithme, basé sur la distance d'édition et la recherche de la plus longue sous-séquence commune (LCS), est opérationnel. Il permet de traiter deux versions d'un texte et d'isoler les changements (ajouts/suppressions) avec une précision chirurgicale.
 - **Intégration de la GUI (Interface Graphique)** : Le travail de **Lucas** et **Maywenn** a permis d'aboutir à une interface stable. Le "binding" entre le code C (Back-end) et l'interface (Front-end) assure une mise à jour de l'affichage en temps réel lors de la saisie.
 - **Gestion des Niveaux de Difficulté** : Le système traite désormais les textes de **Niveau 2** (accords de base) et **Niveau 3** (homophones et contextes complexes), sécurisant ainsi la transition entre un simple prototype et un outil professionnel.
-

4 Difficultés Rencontrées

Le développement de **KOMP** a été jalonné de défis techniques qui ont obligé l'équipe à revoir l'architecture initiale.

1. Gestion de la Mémoire et Segmentation Faults

En travaillant en C pour le moteur de comparaison, la manipulation de chaînes de caractères de tailles dynamiques a été une source majeure d'erreurs.

- **Problème** : Les "leaks" de mémoire lors de la comparaison de fichiers de plus de 10 000 caractères saturaient la pile.
- **Solution** : Mise en place d'une gestion stricte via `malloc` et `free` avec l'utilisation de **Valgrind** pour tracer chaque allocation. Nous avons opté pour une structure de données en "Buffer circulaire" pour optimiser les échanges de texte.

2. Complexité de l'Algorithme de Comparaison

Initialement, l'algorithme de comparaison simple avait une complexité temporelle trop élevée ($O(N \times M)$). Sur de longs textes, l'interface de Lucas freezait pendant plusieurs secondes.

- **Problème** : Temps de calcul inacceptable pour un outil "temps réel".
- **Solution** : **Bayou** à optimiser l'algorithme en implémentant une stratégie de "Heuristique de Myers". Cela a permis de réduire drastiquement le nombre de comparaisons inutiles en ignorant les blocs de texte identiques.

3. Encodage et Caractères Spéciaux (UTF-8)

La gestion des accents français (é, à, ç) et de la ponctuation spéciale a posé des problèmes de reconnaissance au niveau du moteur de correction.

- **Problème** : Un "é" était parfois interprété comme deux caractères distincts, décalant tout le souligne dans la GUI.
- **Solution** : Normalisation systématique des flux d'entrée en **UTF-8** et réécriture des fonctions de comptage de caractères pour qu'elles soient "multi-byte aware".

4. Cohérence Design et Performance (UI/UX)

Le design moderne proposé par **Maywenn** inclut des animations de soulignement complexes qui ralentissaient le rendu.

- **Problème** : Conflit entre l'esthétique et la fluidité de frappe.
 - **Solution** : **Lucas** a implémenté un système de "Debouncing" : le moteur de correction ne se déclenche que 300ms après l'arrêt de la frappe utilisateur, préservant ainsi les ressources processeur
-

5 - Conclusion Générale et Perspectives

Le projet **KOMP** arrive à son terme après un cycle de développement intense et formateur. En partant d'une problématique de traitement de texte complexe, nous avons abouti à une solution logicielle intégrée, performante et ergonomique.

5.1 Synthèse du Travail Réalisé

Nous avons réussi à relever les défis techniques imposés par le cahier des charges initial :

- **Performance** : Le moteur de comparaison et de correction développé en **C** garantit une exécution rapide, même sur des fichiers volumineux de Niveau 3.
- **Ergonomie** : L'interface conçue par **Lucas** et **Maywenn** offre une expérience utilisateur fluide, transformant un outil de calcul brut en une véritable application de bureau.
- **Fiabilité** : Grâce aux optimisations algorithmiques de **Ayoub**, KOMP traite les ambiguïtés grammaticales avec une précision qui dépasse les correcteurs standards noncontextuels.

5.2 Bilan de la Collaboration

Au-delà de l'aspect purement technique, ce projet a été une expérience humaine structurante. La répartition claire des rôles a permis de simuler un environnement de production réel :

- La gestion des dépendances entre le design, le front-end et le back-end a renforcé notre compréhension du **cycle de vie d'un logiciel**.
- L'utilisation de **Git** et de méthodes agiles a été la clé pour surmonter les points de blocage techniques sans impacter le calendrier global.

5.3 Perspectives d'Évolution

Bien que l'application soit totalement opérationnelle, le projet **KOMP** pose des bases solides pour des développements futurs :

- **Apprentissage Machine (IA)** : L'intégration d'un réseau de neurones plus profond pour affiner la détection stylistique.
- **Extension Collaborative** : Un module de révision en temps réel permettant à plusieurs utilisateurs de comparer et corriger un document simultanément.
- **Support Multi-plateforme** : Portabilité de l'interface vers des environnements mobiles ou web pour élargir la base d'utilisateurs.

En conclusion, **KOMP** représente l'aboutissement de nos compétences acquises au cours de ce semestre. Ce projet nous a permis de démontrer qu'une approche rigoureuse "from scratch" est non seulement possible, mais qu'elle permet d'obtenir un contrôle total sur la qualité et la performance du produit final.